COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	10/22/2026	**DATE SUBMITTED:**	10/22/2026
NAME:	Alvarez, Erick Jefferson A	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 **Build a Sample Web App in a Docker Container**

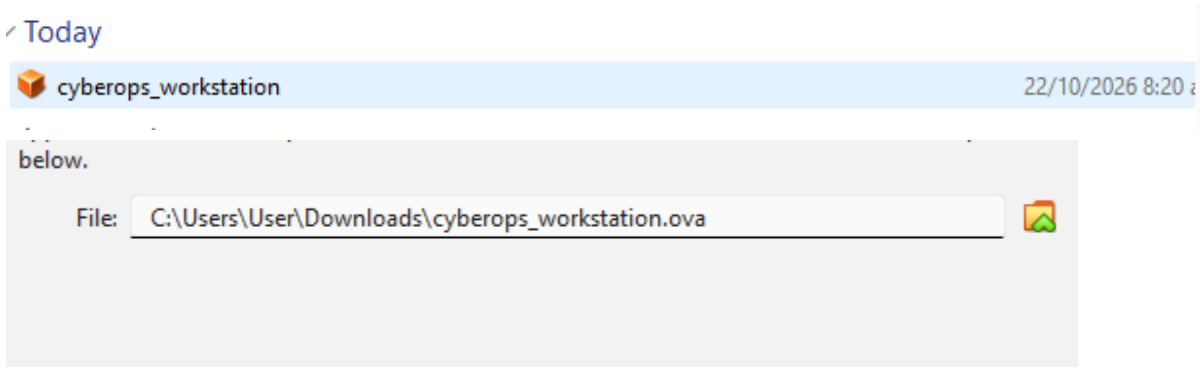
1. PROCEDURE OUTPUT

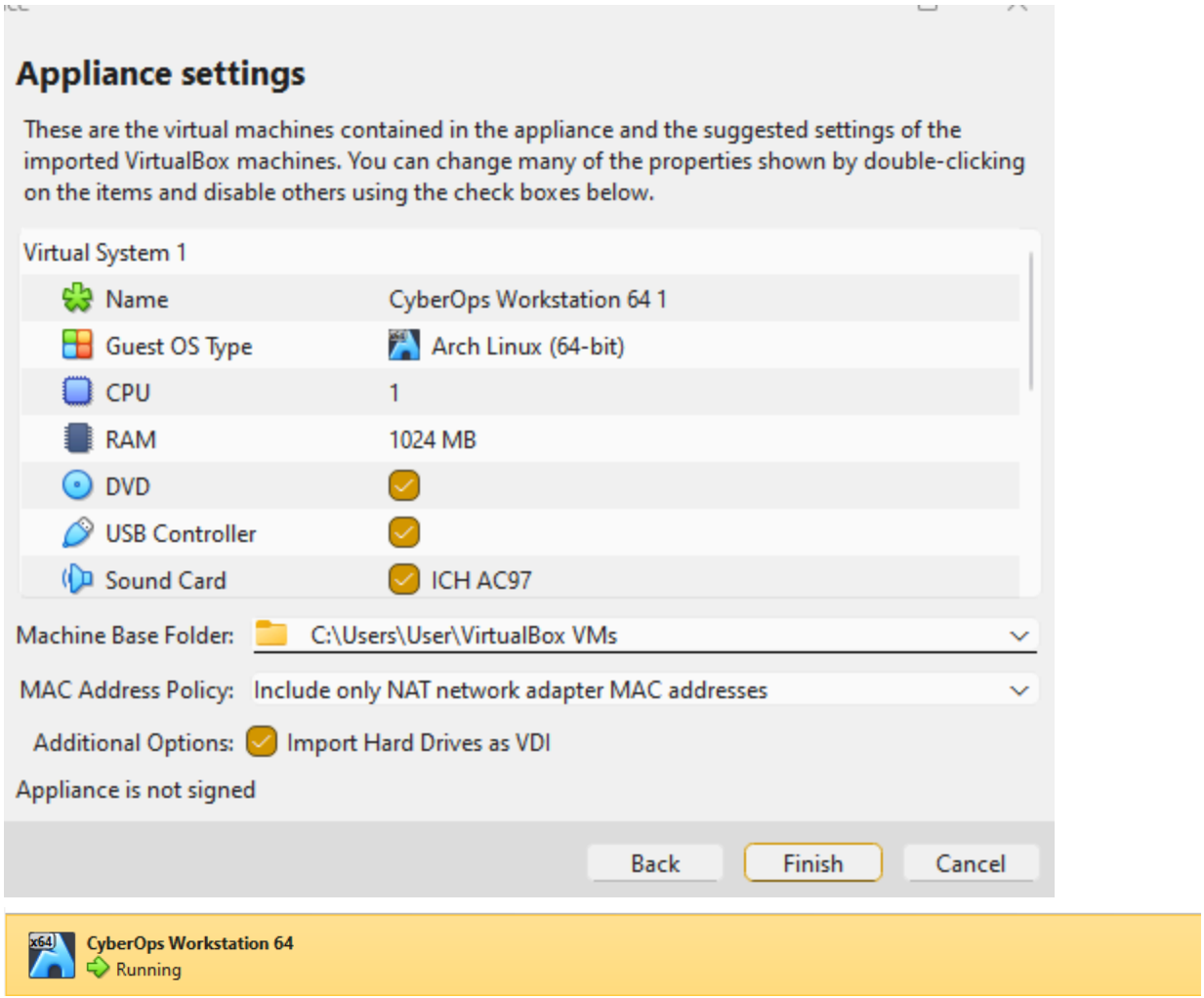
INSTALLATION:

Part 1: Prepare a Host Computer for Virtualization

(VM Already installed)

Part 2: Import the Virtual Machine into the VirtualBox Inventory




Appliance settings
These are the virtual machines contained in the appliance and the suggested settings of the imported VirtualBox machines. You can change many of the properties shown by double-clicking on the items and disable others using the check boxes below.
Virtual System 1
Name
CyberOps Workstation 64 1
Guest OS Type
Arch Linux (64-bit)
CPU
1
RAM
1024 MB
DVD
USB Controller
Sound Card
ICH AC97
Machine Base Folder:
C:\Users\User\VirtualBox VMs
MAC Address Policy:
Include only NAT network adapter MAC addresses
Additional Options:
Import Hard Drives as VDI
Appliance is not signed
Back
Finish
Cancel

CyberOps Workstation 64
Running

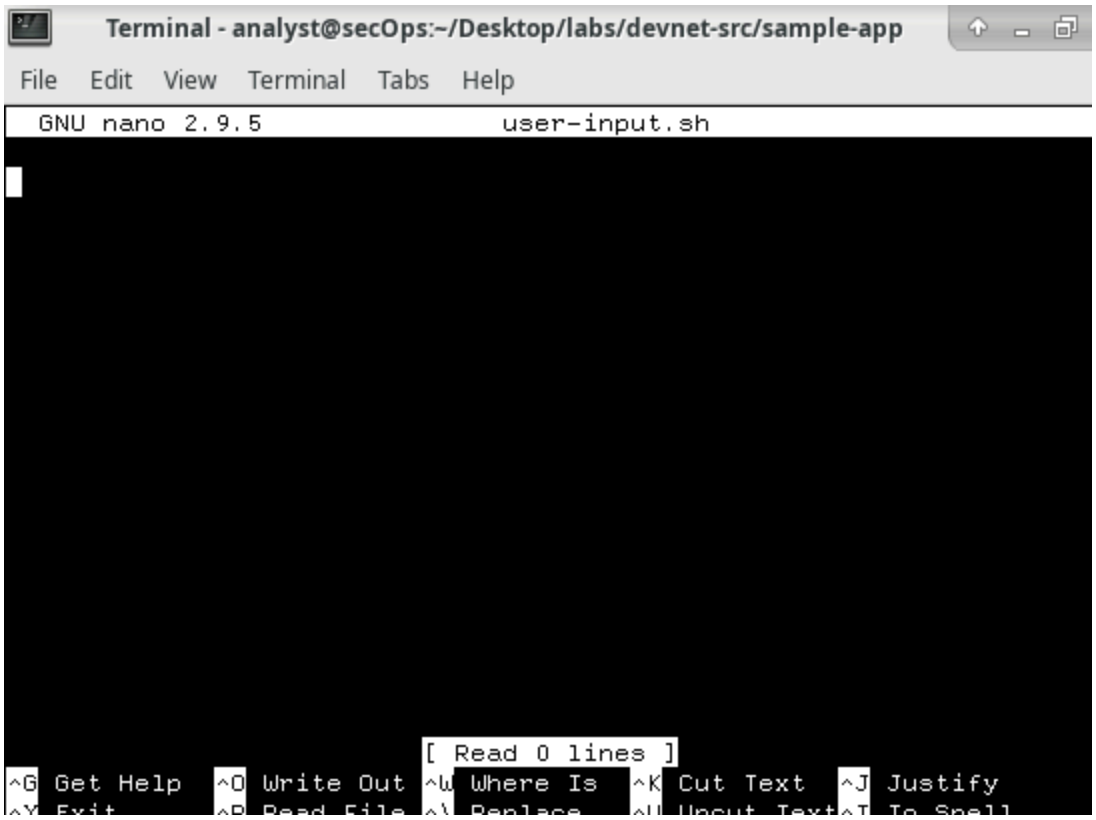


Part 2: Create a Simple Bash Script

Step 1: Create an empty bash script file.

```
[analyst@secOps Desktop]$ mkdir labs
[analyst@secOps Desktop]$ cd labs
[analyst@secOps labs]$ mkdir devnet-src
[analyst@secOps labs]$ ls
devnet-src
[analyst@secOps labs]$ cd devnet-src/
[analyst@secOps devnet-src]$ mkdir sample-app
[analyst@secOps devnet-src]$ cd sample-app/
[analyst@secOps sample-app]$ touch user-input.sh
[analyst@secOps sample-app]$
```

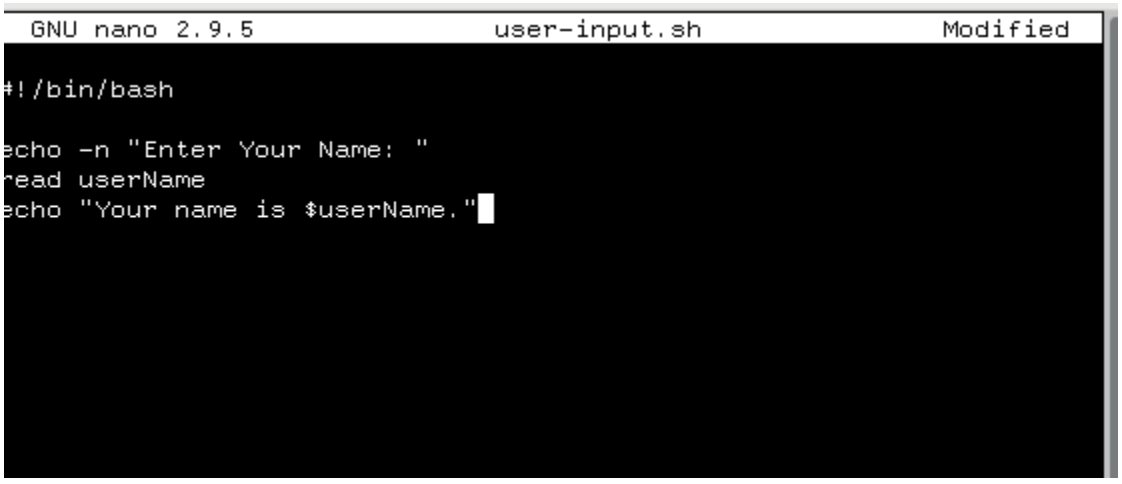
Step 2: Open the file in the nano text editor



Step 3: Add the 'she-bang' to the top of the script.



Step 4: Add simple bash commands to the script



Step 5: Exit nano and save your script.

```
[analyst@secOps devnet-src]$ cd sample-app/
[analyst@secOps sample-app]$ ls
user-input.sh
[analyst@secOps sample-app]$ nano user-input.sh
[analyst@secOps sample-app]$
```

Step 6: Run your script from the command line.

```
[analyst@secOps sample-app]$ nano user-input.sh
[analyst@secOps sample-app]$ bash user-input.sh
Enter Your Name: Erick
Your name is Erick.
[analyst@secOps sample-app]$
```

Step 7: Change the mode of the script to an executable file for all users

```
[analyst@secOps sample-app]$ ls -l user-input.sh
-rw-r--r-- 1 analyst analyst 86 Oct 21 21:15 user-input.sh
[analyst@secOps sample-app]$ chmod a+x user-input.sh
[analyst@secOps sample-app]$ ls -l user-input.sh
-rwxr-xr-x 1 analyst analyst 86 Oct 21 21:15 user-input.sh
[analyst@secOps sample-app]$
```

Step 8: Rename the file to remove the .sh extension

```
[analyst@secOps sample-app]$ mv user-input.sh user-input
[analyst@secOps sample-app]$ ls
user-input
[analyst@secOps sample-app]$
```

Step 9: Execute the script from the command line.

```
[analyst@secOps sample-app]$ ./user-input
Enter Your Name: Erick
Your name is Erick.
[analyst@secOps sample-app]$
```

Step 10: Investigate other bash scripts.

```
  GNU nano 2.9.5                guessthenumber.sh

#!/bin/bash

secret=$(( RANDOM % 100 + 1 ))
guess = 0

echo "Guess a number between 1 and 100:"
while [ %guess -ne %secret ]; do
        read guess
        if [ %guess -lt $secret ]; then
                echo "Too low!"
        elif [ $guess -gt $secret ]; then
                echo "Too high!"
        else
                echo "That's right!"
        fi
done
```

```
Too low!
81
Too high!
80
Too high!
79
Too high!
75
Too high!
74
Too high!
73
That's right!
[analyst@secOps sample-app]$
```

Part 3: Create a Sample Web App

Step 1: Install Flask and open a port on the DEVASC VM firewall.

```
WARNING: Running pip as the 'root' user can result in broken p
ons and conflicting behaviour with the system package manager.
recommended to use a virtual environment instead: https://pip.
/warnings/venv
[analyst@secOps sample-app]$ flask --version
Python 3.6.5
Flask 2.0.3
Werkzeug 2.0.3
[analyst@secOps sample-app]$
```

Step 2: Open the sample_app.py file

```
[analyst@secOps sample-app]$ nano sample_app.py
[analyst@secOps sample-app]$ ls
guessthenumber.sh  sample_app.py  user-input
[analyst@secOps sample-app]$
```

Step 3: Add the commands to import methods from flask.

```
  GNU nano 2.9.5                    samp

from flask import Flask
from flask import request
```

Step 4: Create an instance of the Flask class.

```
sample = Flask(__name__)
```

Step 5: Define a method to display the client IP address.

```
sample = Flask(__name__)

@sample.route("/")
def main():
        return "You are calling me from" + rquest.remote_addr
+ "/n"
```

Step 6: Configure the app to run locally

```
if __name__ == "__main__":
        sample.run(host="0.0.0.0", port=8080)
```

Step 7: Save and run your sample web app.

```
 * Serving Flask app 'sample_app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a producti
on deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on all addresses.
   WARNING: This is a development server. Do not use it in a producti
on deployment.
 * Running on http://10.0.2.15:8080/ (Press CTRL+C to quit)
```

Step 8: Verify the server is running.

```
   WARNING: This is a development server. Do not use it in a pr
on deployment.
 * Running on http://10.0.2.15:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [21/Oct/2026 21:56:01] "GET / HTTP/1.1" 200 -
```

Step 9: Stop the server

```
   WARNING: This is a development server. Do not use it in a produc
on deployment.
 * Running on http://10.0.2.15:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [21/Oct/2026 21:56:01] "GET / HTTP/1.1" 200 -
^C[analyst@secOps sample-app]$
```

Part 4: Configure the Web App to use Website Files

Step 1: Explore the directories that will be used by the web app.

```
File  Edit  View  Search  Terminal  Help
devasc@labvm:~/labs/devnet-src/sample-app/templates$ cat index.html
<html>
<head>
    <title>Sample app</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>You are calling me from {{request.remote_addr}}</h1>
</body>
</html>
devasc@labvm:~/labs/devnet-src/sample-app/templates$
```

```
devasc@labvm: ~/labs/devnet-src/sample-app/templates
File  Edit  View  Search  Terminal  Help
 GNU nano 4.8                                  index.html
<html>
<head>
    <title>Devastating month</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>I miss {{request.remote_addr}}</h1>
</body>
</html>
```

Step 2: Update the Python code for the sample web app

```
devasc@labvm: ~/labs/devnet-src/sample-app
File  Edit  View  Search  Terminal  Help
 GNU nano 4.8                                sample_app.py
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

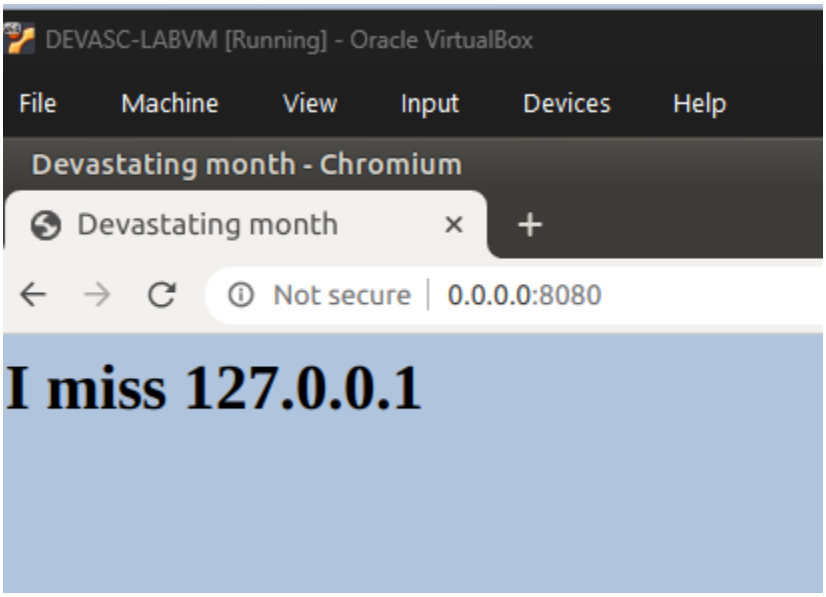
@sample.route("/")
def main():
        return render_template("index.html")

if __name__ == "__main__":
        sample.run(host="0.0.0.0", port=8080)
```

Step 3: Save and run your script.

```
devasc@labvm:~/labs/devnet-src/sample-app$ nano sample
_app.py
devasc@labvm:~/labs/devnet-src/sample-app$
```

Step 4: Verify your program is running.



Step 5: Stop the server.

```
127.0.0.1 - - [22/Oct/2026 11:27:09] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Oct/2026 11:27:09] "GET /static/style.css HTTP/1.1" 200 -
127.0.0.1 - - [22/Oct/2026 11:27:09] "GET /favicon.ico HTTP/1.1" 404 -
^Cdevasc@labvm:~/labs/devnet-src/sample-app$
```

Part 5: Create a Bash Script to Build and Run a Docker Container

Step 1: Create temporary directories to store the website files.

```
  GNU nano 4.8
#!/bin/bash

mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static
```

Step 2: Copy the website directories and sample_app.py to the temporary directory.

```
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.
```

Step 3: Create a Dockerfile

a.
```
echo "FROM python" >> tempdir/Dockerfile
```

b.
```
echo "FROM python" >> tempdir/Dockerfile
echo "RUN pip install flask" >> tempdir/Dockerfile
```

c.
```
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfi
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
```

d.
```
echo "EXPOSE 8080" >> tempdir/Dockerfile
```

e.
```
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile
```

Step 4: Build the Docker container.

```
cd tempdir
docker build -t sampleapp .

^G Get Help  ^O Write Out ^W Where
^X Exit      ^R Read File ^\ Replac
```

Step 5: Start the container and verify it is running.

a.
```
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify
```

b.
```
docker ps -a
```

Step 6: Save your bash script

```
devasc@labvm:~/labs/devnet-src/sample-app$ nano sample-app.sh
devasc@labvm:~/labs/devnet-src/sample-app$
```

Part 6: Build, Run, and Verify the Docker Container

Step 1: Execute the bash script.

```
 ---> 46a5c1b18ac9
Successfully built 46a5c1b18ac9
Successfully tagged sampleapp:latest
febc30d47665ba0c2594821157c29b40ee9fef96f49b454944fc2d7d7a862f87
CONTAINER ID        IMAGE               COMMAND                  CREATED
     STATUS                  PORTS                    NAMES
febc30d47665        sampleapp           "/bin/sh -c 'python3…"   1 second ago
     Up Less than a second   0.0.0.0:8080->8080/tcp   samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$
```

Step 2: Investigate the running Docker container and the web app.

a.
```
     Exited (2) 5 minutes ago                                  jovi
devasc@labvm:~/labs/devnet-src/sample-app$ ls tempdir/
Dockerfile  sample_app.py  static  templates
devasc@labvm:~/labs/devnet-src/sample-app$
```

b.

```
devasc@labvm:~/labs/devnet-src/sample-app$ cat tempdir/Dockerfile
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
devasc@labvm:~/labs/devnet-src/sample-app$
```

c.

```
CMD python3 /home/myapp/sample_app.py
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a >> running.t
devasc@labvm:~/labs/devnet-src/sample-app$
```

d.

```
devasc@labvm:~$ curl http://127.0.0.1:8080
<html>
<head>
    <title>Devastating month</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>I miss 127.0.0.1</h1>
</body>
</html>devasc@labvm:~$
```

e.

```
^Cdevasc@labvm:~/labs/devnet-src/sample-app$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN gro
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel st
oup default qlen 1000
    link/ether 08:00:27:e9:3d:e6 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
N group default
    link/ether 02:42:4f:18:e7:40 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
    inet6 fe80::42:4fff:fe18:e740/64 scope link
       valid_lft forever preferred_lft forever
```

Step 3: Access and explore the running container.

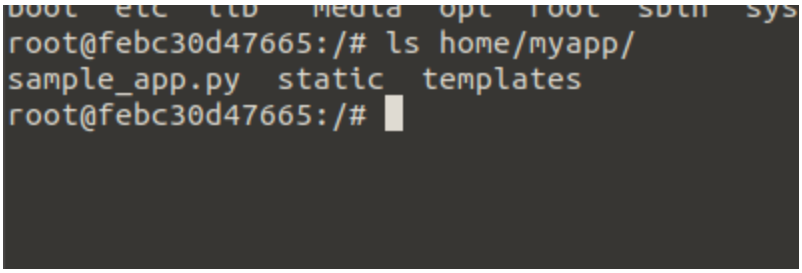
a.

```
devasc@labvm: ~/labs/devnet-src/sample-app
File  Edit  View  Search  Terminal  Help
devasc@labvm:~/labs/devnet-src/sample-app$ docker exec -it samplerunning /bi
sh
root@febc30d47665:/#
```

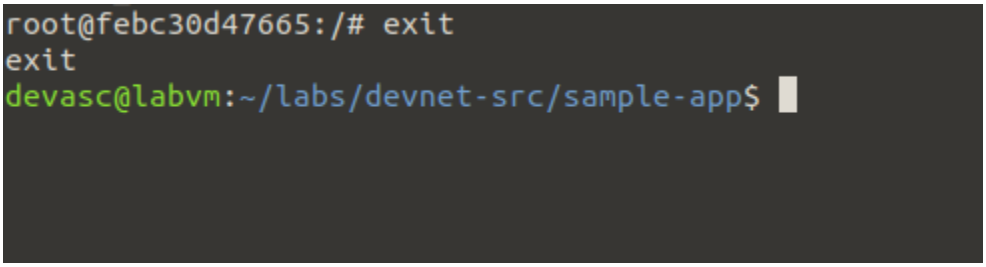
b.

```
root@febc30d47665:/# ls
bin   dev  home  lib64  mnt  proc  run   srv  tmp  var
boot  etc  lib   media  opt  root  sbin  sys  usr
root@febc30d47665:/#
```

c.

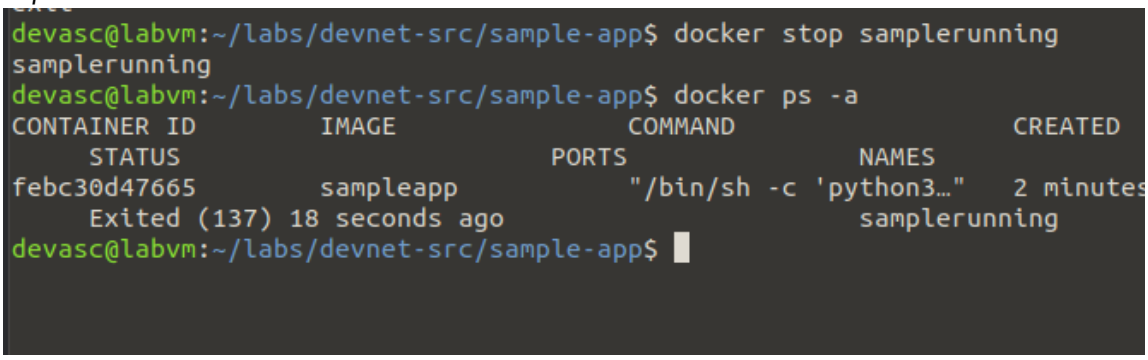


d.

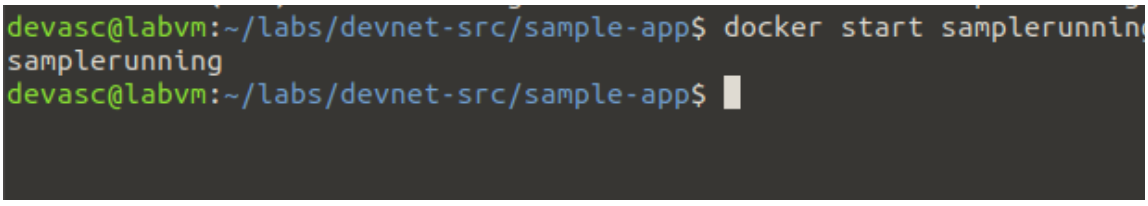


Step 4: Stop and remove the Docker container.

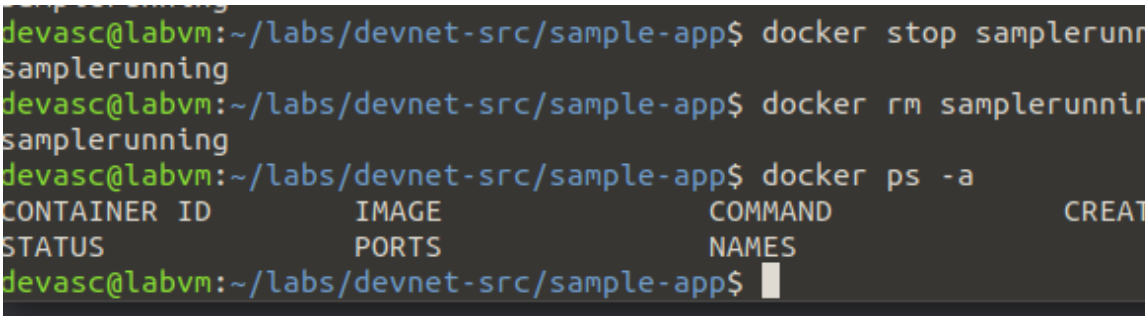
a.



b.



c.



Analysis:

Parts	Analysis
Part 1: Installation (Launch the DEVASC VM)	Starting the DEVASC VM gave me a ready-made Linux environment with Docker, Python, and the lab files already installed. Setting all of this up by hand on my own computer would have taken much longer, and differences between operating systems could have caused problems. Using the VM kept the environment the same for every part of the lab, so when something broke later, I knew the setup itself wasn't the cause.
Part 2: Create a Simple Bash Script	I learned that a bash script is just a list of terminal commands saved in a file so they can be run again. The shebang line tells the system which interpreter to use. A new file couldn't run until I gave it execute permission with chmod, which showed me how Linux permissions control what a file can do. This practice mattered because the later parts rely on one script to automate the whole Docker process.
Part 3: Create a Sample Web App	I built a small web app with Flask. The code needed only a few lines: import Flask, create an app instance, define a route, and run the server. The route returned the IP address of whoever visited the page. Opening the port in the firewall showed me that a server can run correctly and still be unreachable until the network allows traffic to it. Stopping the server also showed that the app only exists while its process is running.
Part 4: Configure the Web App to Use Website Files	I changed the app to serve an HTML page from a templates folder and a stylesheet from a static folder, instead of returning plain text. This separates the app's logic from the way it looks. Flask expects those specific folder names, so a project that doesn't follow the structure won't find its files. I had to copy that same structure exactly when packaging the app into a container.
Part 5: Create a Bash Script to Build and Run a Docker Container	This part brought the earlier work together. The script creates a temporary directory, copies in the app and website files, writes a Dockerfile line by

	line with echo, builds the image, and runs it as a container. The Dockerfile works like a recipe: it names the base image, installs Flask, copies the files, opens the port, and sets the start command. The script also has a weak point. It appends to the Dockerfile with >> and doesn't clear the old folder first, so running it more than once stacks up duplicate lines.
Part 6: Build, Run, and Verify the Docker Container	This was where I hit real problems. The build failed at RUN pip install flask with a "can't start new thread" error, and docker run then said the image couldn't be found. That second error was only a side effect, because the failed build never created an image. The cause was pip's progress bar trying to start a background thread in an environment that wouldn't allow it, along with a very new pre-release Python base image. I fixed it by pinning the base image to python:3.12-slim and turning off the progress bar. I also found that a leftover tempdir had inflated the Dockerfile to 49 steps instead of 7, so I added rm -rf tempdir to the start of the script. After the build worked, I confirmed the container was running and the app responded on port 8080, then stopped and removed it. The main lesson was to read the first error in the output instead of the last one. It also showed me why containers are useful: pinning the exact Python version makes the app behave the same way every time.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CONCLUSION:
This lab illustrated how an idea translates into a real experiment in a step-by-step way. My lab began with several bash commands and a mini-Flask application, and moved further to create a Docker image, which could be made and run with a single command line. The next step was depending on previous steps, so not performing one of them properly, or performing in hurry would complicate things at all.

That part of the lab, where I had the greatest learning experience was Part 6 (troubleshooting). It was frustrating to face the errors during the builds and to find out that the last error of build was not the reason of all the problems. Thanks to reading carefully output messages from my previous attempts to build and to understand, what exactly my script is doing in this case, I discovered the problem. Also, now I know that any script should perform without creating unnecessary files.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error explanation and debugging**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used only to understand and fix the error.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.